Agile Software Development

References:
https://www.agilealliance.org/
https://agilemanifesto.org/



Activity

Why do software requirements change during development?

What is Agile ?

- Agile is the ability to create and respond to change.
- It is a way of dealing with, and ultimately succeeding in, an uncertain and turbulent environment.
- It's really about thinking through how you can understand what's going on in the environment that you're in today, identify what uncertainty you're facing, and figure out how you can adapt to that as you go along.
- Agile is an iterative and incremental software development approach where software is developed in small, manageable releases, allowing teams to quickly respond to changing customer requirements.
- Instead of delivering everything at the end,
- Agile delivers working software continuously.

What is Agile contd…?

- Agile software development is more than frameworks such as Scrum, Extreme Programming, or Feature-Driven Development (FDD).
- Agile software development is more than practices such as pair programming, test-driven development, stand-ups, planning sessions, and sprints.
- Agile software development is an umbrella term for a set of frameworks and practices based on the values and principles expressed in the Manifesto for Agile Software Development and the 12 Principles behind it.
- When we approach software development in a particular manner, it's generally good to live by these values and principles and use them to help figure out the right things to do given our particular context.
- In Agile software development, solutions evolve through collaboration between self-organizing cross-functional teams utilizing the appropriate practices for their context.

What is Agile Software Development?

Agile is a Mindset.

Agile is a mindset shaped by the Agile Manifesto's values and principles.

Those values and principles provide guidance on how to create and respond to change and how to deal with uncertainty.

Agile Mindset

- Agile is a Mindset
- Agile is not just a methodology.
- It is a way of thinking that emphasizes on
  - Customer collaboration
  - Teamwork
  - Continuous improvement
  - Flexibility
  - Fast delivery
  - Learning from feedback

Key Agile Practices

Iterative Development

Work in short cycles (Sprints or Iterations)

Incremental Delivery

Each iteration delivers usable software

Daily Stand-ups

Short meetings to discuss progress, blockers, and next steps.

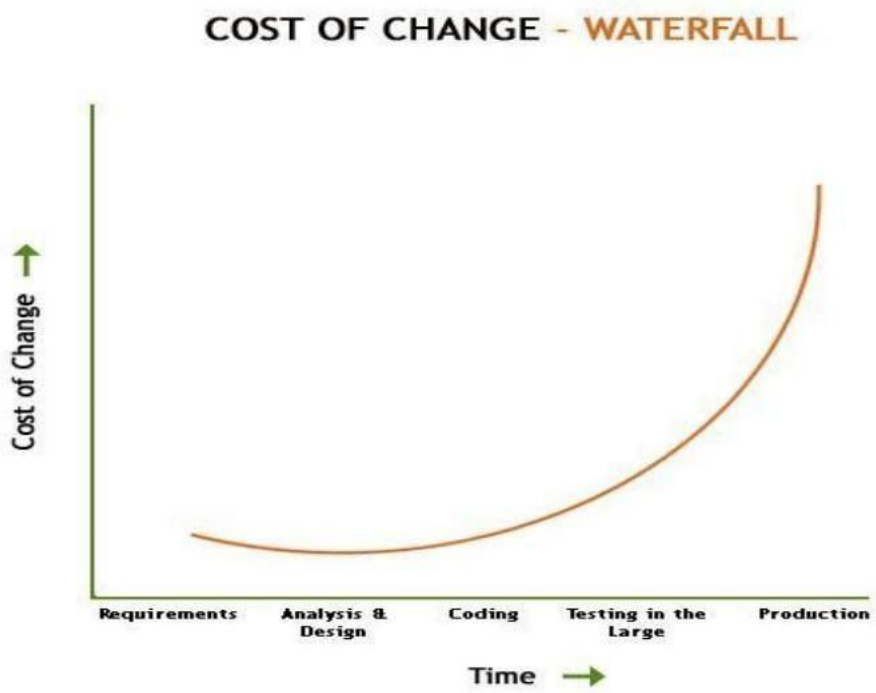
User Stories

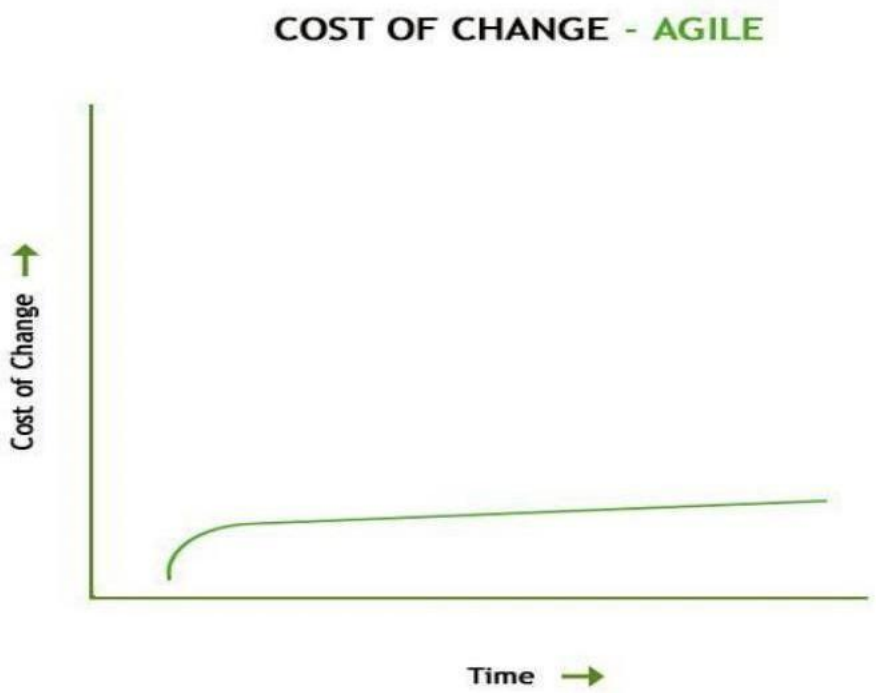
Requirements written from the user perspective (e.g., *“As a user, I want to reset my password so that I can access my account if I forget it.”*).

Continuous Feedback

Incorporate user feedback every iteration.

Why Agile ?


COST OF CHANGE - WATERFALL
Cost of Change
Requirements
Analysis & Design
Coding
Testing in the Large
Production
Time


COST OF CHANGE - AGILE
Cost of Change
Time

Agile Values

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Agile Manifesto

Agile Principles

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.



Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.



Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Business people and developers must work together daily throughout the project.

Agile Principles

Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.

The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.



Working software is the primary measure of progress.

Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

Continuous attention to technical excellence and good design enhances agility.

Agile Principles

Simplicity- the art of maximizing the amount of work not done is essential.

The best architectures, requirements, and designs emerge from self-organizing teams.

At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.



Agile Scrum

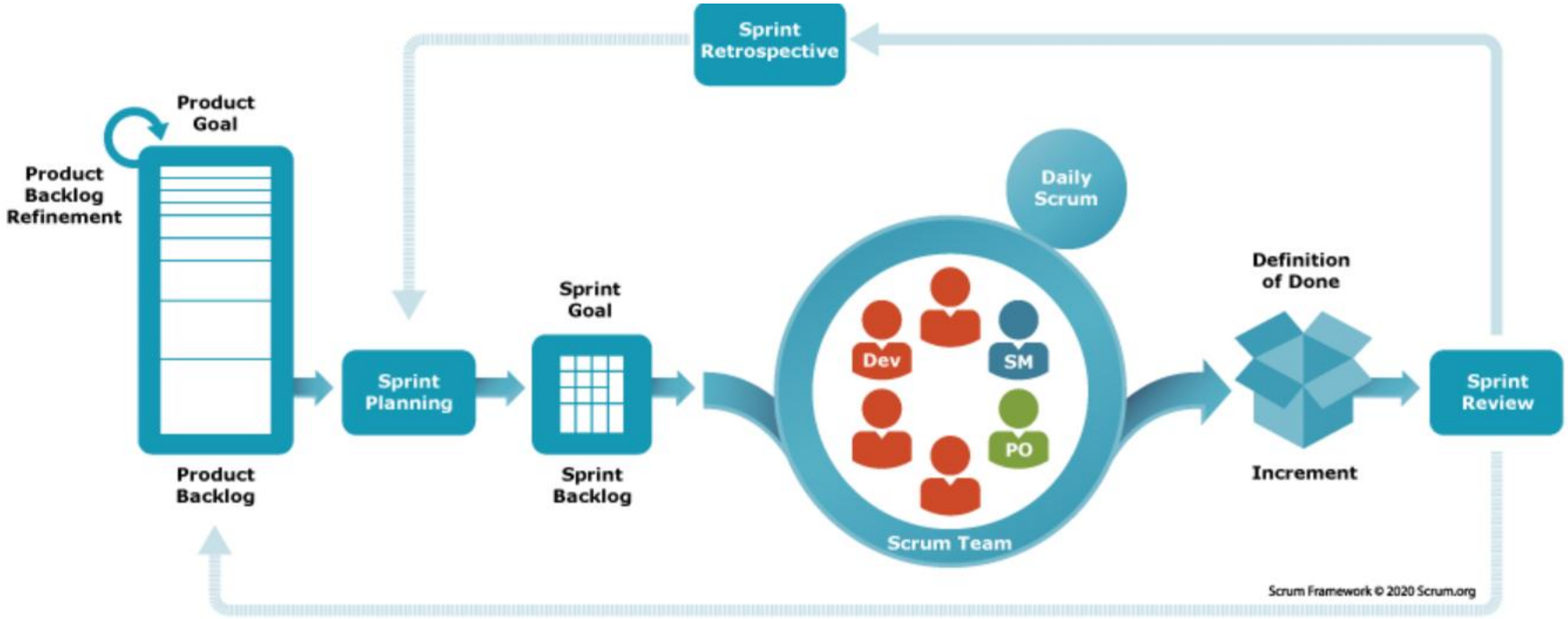
Agile Scrum

- Scrum is a lightweight Agile framework used to manage and develop complex projects, especially software development projects.
- It helps teams work together efficiently by breaking a large project into small, manageable pieces of work called Sprints.
- Instead of trying to build the entire product at once, Scrum focuses on delivering working software in short, fixed time periods, while continuously improving based on customer feedback.

Sprint

- Sprint is the basic unit of work for a Scrum team.
- Iterative cycles
- A sprint is a short, time-boxed period when a scrum team works to complete a set amount of work.
- As a rule, a sprint lasts about 1-4 weeks.
- This is the main feature that marks the difference between Scrum and other models for agile development.

Agile Life Cycle



Scrum Team

- One Scrum Master
- One Product Owner
- Developers
- Scrum Teams are cross-functional.
- They are also self-managing.
- The Scrum Team is small enough to remain nimble and large enough to complete significant work within a Sprint, typically 10 or fewer people.
- The Scrum Team is responsible for all product-related activities from stakeholder collaboration, verification, maintenance, operation, experimentation, research and development, and anything else that might be required.

Developers

Developers are the people in the Scrum Team that are committed to creating any aspect of a usable Increment each Sprint.

The Developers are always accountable for;

- Creating a plan for the Sprint, the Sprint Backlog;
- Instilling quality by adhering to a Definition of Done;
- Adapting their plan each day toward the Sprint Goal
- Holding each other accountable as professionals.

Product Owner

- The Product Owner is accountable for maximizing the value of the product resulting from the work of the Scrum Team.
- The Product Owner is also accountable for effective Product Backlog management, which includes:
  - Developing and explicitly communicating the Product Goal
  - Creating and clearly communicating Product Backlog items
  - Ordering Product Backlog items
  - Ensuring that the Product Backlog is transparent, visible and understood.
  - The Product Owner is one person, not a committee.
- He may represent the needs of many stakeholders in the Product Backlog.
- Those wanting to change the Product Backlog can do so by trying to convince the Product Owner.

Scrum Master

- The Scrum Master is accountable for establishing Scrum as defined in the Scrum Guide.
- They do this by helping everyone understand Scrum theory and practice, both within the Scrum Team and the organization.
- The Scrum Master is accountable for the Scrum Team’s effectiveness.
- They do this by enabling the Scrum Team to improve its practices, within the Scrum framework. Scrum Masters are true leaders who serve the Scrum Team and the larger organization.

Sprint Life Cycle

Each Sprint involves

- Sprint planning
- Sprint preparation
- Daily Activities
- Sprint Demo
- Sprint Retrospective

Sprint Preparation (Backlog Refinement)

- Prepare Product Backlog before Sprint Planning.
- Not an official Scrum event.
- Activities
  - Review Product Backlog Items
  - Clarify requirements
  - Split Epics into User Stories
  - Estimate using Story Points
  - Prioritize backlog

Example - Backlog Refinement

Epic: Build Online Shopping System

- Registration (3 Story Points [SP])
- Login (2 SP)
- Search Products (5 SP)
- Shopping Cart (8 SP)
- Checkout (8 SP)

User stories now ready for Sprint Planning.

Sprint Planning

Official Scrum event starting every Sprint.

Participants: Product Owner, Scrum Master, Developers.

Topic 1: Why is this Sprint valuable?

- Define Sprint Goal.

Topic 2: What can be completed?

- Select Product Backlog Items.

Topic 3: How will work be done?

- Break stories into tasks.

Outputs: Sprint Goal + Sprint Backlog.

Sprint Planning Example



Sprint Goal: Customers can browse and purchase products.



Selected Stories

Login
Registration
Search Products



Tasks

Design UI
Develop APIs
Testing

Daily Scrum



15-minute meeting on every working day.



Purpose is to inspect progress toward Sprint Goal.



Developers participate.



Typical discussion

Tasks completed yesterday
Today's work
Blockers



Outcome: Updated plan for next 24 hours.

Development During Sprint

- UI Design
- Coding
- Database Development
- Code Reviews
- Unit Testing
- Integration Testing
- Bug Fixing
- Backlog updated as new learning occurs.

Sprint Review (Sprint Demo)



Held at the end of the Sprint.



Purpose: Inspect completed Increment.



Participants

- Scrum Team
- Stakeholders
- Customers



Activities

- Demonstrate working software
- Collect feedback
- Update Product Backlog

Sprint Review Example

Demo: Login, Registration and Search completed.

Stakeholder requests Google Login.

New request added to Product Backlog.

Sprint Retrospective



Final Scrum event of the Sprint.



Purpose: Improve team effectiveness.



Discuss

What went well?
What went wrong?
What should improve?

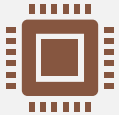


Output: Improvement actions for next Sprint.

Retrospective Example



Issue: Testing delayed.



Cause: Testing started too late.



Improvement: Write automated tests during development.



Action added to next Sprint.

Sprint Review vs Sprint Retrospective

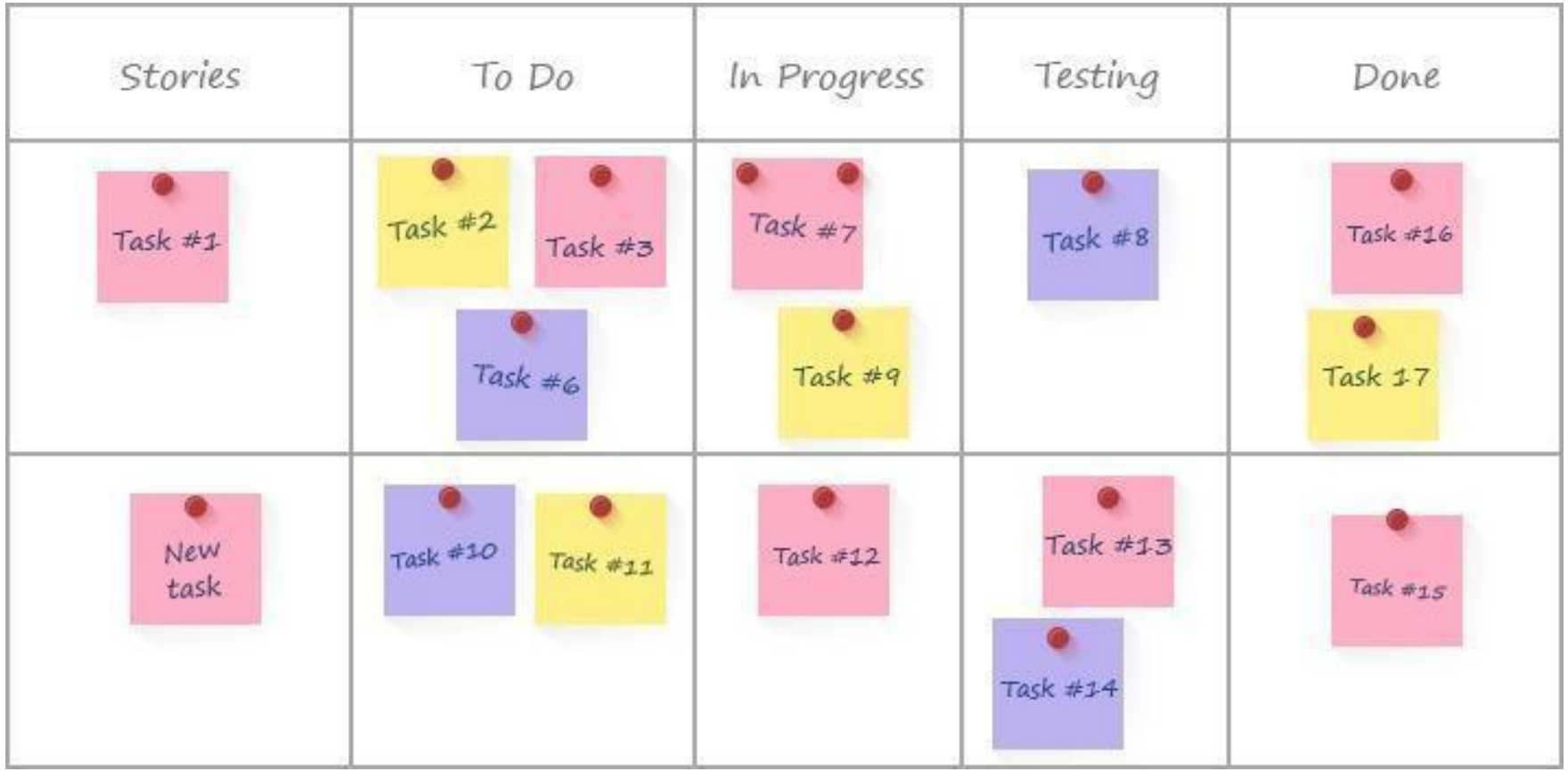
Sprint Review: Product-focused.

- Stakeholders attend.
- Collect feedback.

Sprint Retrospective: Process-focused.

- Only Scrum Team.
- Improve teamwork and practices.

Sprint Board Example



Key Agile Concepts

- Epics
- User Stories
- Tasks
- Story Points

Epics

- Epics are large user stories, typically ones which are too big to implement in a single iteration and therefore they need to be disaggregated into smaller user stories.
- Epic can be broken down into user stories.

Eg: As a customer, I want to be able to buy products from the online store so that I can save my time.

Epic-Level Planning

- Break epics into smaller user stories.
- Estimate user stories individually.

Eg:

Epic: Online Shopping Platform

Stories: Search Products, Add to Cart, Checkout.

User Stories

- An informal, general explanation of a software feature written from the perspective of the end user.
- Its purpose is to articulate how a software feature will provide value to the customer.
- A user story is the smallest unit of work in an agile framework.

Eg:

As a customer, I want to be able to browse the online store so that my time is saved.

User Stories

- Use the following template in writing user stories.

As a <role>,

I can <activity>

so that <business value>

.

.

Tasks

- Steps/work needed to implement a user story.
- Eg: 'Create Add to Wishlist button and backend API.’
- A user story can have multiple tasks.

Story Points

- Relative measure of effort, complexity, and risk.
- Estimate effort for user stories.
- Factors such as complexity, workload, risk, dependencies/complexities are considered.
- Relative scale, not time-based.

Size = Effort + Risk + Complexity

- Eg:

Simple login page = 2 points, full-feature checkout = 8 points.

Story Points - Velocity-Based Forecasting

Velocity = average number of story points a team completes in a sprint (e.g., 2 weeks).

Example:

- Sprint 1: 25 points delivered
- Sprint 2: 22 points delivered
- Sprint 3: 27 points delivered
- Average Velocity ≈ **25 points per sprint**

If your sprint is 2 weeks (10 working days), then **25 points ≈ 10 working days** for the whole team.

So, **1 story point ≈ 10/25 = 0.4 days of team effort** (but not per individual, it's for the team collectively).

Example of Story Point Estimation

User Story: Upload profile picture

- Dev A: 2 points (simple)
- Dev B: 5 points (with validations)
- Dev C: 8 points (risks with image processing)
- Final agreed estimate: 5 points

Benefits of Story Points and Estimation

- Encourages team discussion and shared understanding.
- Focus on relative complexity, not hours.
- Helps forecast sprint workload (velocity).
- Supports release planning and prioritization.

Example: Sprint of 20 points → plan which stories can be included.

Summary



Story Points Are NOT Time



Not hours or days.

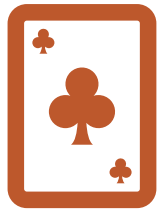


Measure relative effort.



Consider complexity, uncertainty, effort and risk.

Scrum Estimation Techniques



- Planning Poker



- T-Shirt Sizing



- Triangular Average

Planning Poker

- Also called Scrum Poker.
- A popular consensus-based estimation technique used in Agile teams to assign story points to user stories.

Planning Poker Process

1. Product Owner presents a story.
2. Team asks questions.
3. Members select story point cards privately.
4. Reveal simultaneously.
5. Discuss large differences and reach consensus.

Eg: 'Add to Wishlist' estimates: Dev A: 2, Dev B: 3, Dev C: 5 → Final: 3 points.

References: https://www.youtube.com/watch?v=gE7srp2BzoM

https://planningpokeronline.com/Ti0ZOnFVJiqJQpm9mmU1/

Activity

As a user, I want to reset my password via email so that I can regain access if I forget it.

T Shirt Sizing

- Often used when teams want quick, relative estimates without diving into numbers right away.
- It’s a way to size user stories (or even epics) using relative categories instead of story points.
  - XS (Extra Small)
  - S (Small)
  - M (Medium)
  - L (Large)
  - XL (Extra Large)
  - Some teams also use XXL for very large epics.

T Shirt Sizing Process

- **Discuss the story/epic** → The Product Owner explains the requirements.
- **Relative sizing** → The team compares the story with previously completed ones.
- **Assign a size** → Based on complexity, risk, effort, and unknowns.
- **Convert later if needed** → Sizes can be mapped to story points later

(e.g., S = 2 points, M = 5 points, L = 8 points).

Example

- Login screen with username/password” → S (fairly simple).
- “Password reset with email verification” → M (needs more effort, integration).
- “Shopping cart with payment gateway integration” → L (security, APIs, testing).
- “Full recommendation engine for users” → XL (lots of unknowns, heavy complexity)

Triangular Average

$$\text{Weighted average} = \frac{(\text{Optimistic estimate} + 4 \times \text{Most Likely estimate} + \text{Pessimistic estimate})}{6}$$

Definition of Done (DoD)

- Shared understanding within the team of what “done” means for a user story, feature, or product increment.
- A quality checklist agreed upon by the team to ensure consistency and avoid misunderstandings.

Why DoD is Important?

- Ensures clarity: Everyone knows when work is really finished.
- Ensures quality: Stories meet the same standards each sprint.
- Prevents technical debt: Avoids half-done or buggy features slipping through.
- Builds trust: Product Owner and stakeholders can rely on the increment.

Typical Items in a DoD

- Code is written, reviewed, and checked into version control.
- Unit tests are written and passed.
- Integration tests passed.
- Feature is deployed in a staging environment.
- Documentation (user guide / technical) updated.
- Meets acceptance criteria in the user story.
- No open critical bugs.
- Approved by Product Owner / QA.

Example

User story: “As a user, I can reset my password via email so that I have a backup option.”

This is Done when,

- The reset link works in staging.
- Email is sent with token (verified).
- Token expires after 5 minutes.
- Unit tests and integration tests written and passed.
- Documentation updated.
- Stakeholder tested and accepted.

Summary



• Agile estimation combines epics, user stories, tasks, and story points.



• Use Planning Poker, T-Shirt sizing, and Triangular Average for accuracy.



• Clear Definition of Done ensures quality.



• Distinguishing stories from tasks improves tracking and accountability.

Self Study - Extended Scrum Artifacts

- Burndown chart
- Story points